DOMS Page No.
Date / /

ASSIGNMENT - 11

Problem Statement: Write a program in C to accept a number from user & compute the following
a) Square root of number
b) Square of the number.
c) Cube of the number
d) Check whether the number is prime.
e) factorial of number
f) Prime factors of number.

Objective:
- To understand the use of mathematical operations in C
- To implement loops & conditional statements
- To perform number-based computations
- To enhance logical thinking using menu-driven programs

Theory: Numbers & their mathematical properties play a vital role in programming. Operations such as square, cube, and square root are basic mathematical computations. Prime number checking involving verifying whether a number has exactly two divisors.

Platform: Linux

Algorithm:
Step1: START.
Step2: Declare variables n, i, count, fact = 1.
Step3: Read the integer n from user
Step4: Calculate square root of n using sqrt(n).
Step5: Calculate square of n as n * n.

Step6: Calculate cube of n as $n \times n \times n$

Step7: Check whether the no. is prime.

a) If $n <= 1$, it is not prime.

b) Else, check divisibility of n from 2 to $n/2$

Step8: Calculate fact. of n: a) Initialise fact = 1

b) Multiply no's from 1 to n.

Step9: Find prime factors of n: a) for i = 2 to n,

b) while n % i == 0, print i & divide n by i.

Step10: Display all results

Step11 STOP.

C Program:

```
# include <stdio.h>
# include <math.h>
int main() {
    int n, i, flag = 1;
    long long fact = 1;
    printf("Enter a num:");
    scanf("%d", &n);
    printf("Square root : %.2f \n", sqrt(n));
    printf("Square: %d \n", n*n);
    printf("Cube: %d \n", n*n*n);
    if (n <= 1) {
        flag = 0;
    } else {
        for (i = 2; i <= n/2 ; i++) {
            if (n % i == 0) {
                flag = 0;
                break;
            }
        }
}
```

DOMS Page No.
Date / /

```
if(flag==1)
    printf("num is Prime\n");
else
    printf("num is Not Prime\n");
for(i=1; i<=n; i++){
    fact=fact*i;
}
    printf("Prime factors:");
    for(i=2; i<=n; i++){
        while(n%i==0){
            printf("%d", i);
            n=n/i;
        }
    }
return 0;
}
```

Input: The user enters an integer number & selects an operation from the menu.
Example input: Number: 5 choice: 3

Output: The program displays the result based on selected operation.

Example Output: Cube = 125

Date / / Page No.

Conclusion: Thus, the program to perform various mathematical operations on a number was successfully implemented using C programming.

FAQs:

1. Why is checking divisibility only up to n/2 sufficient in prime number verification, and how can this be optimized further?

A. a) Any factor of n must be less than or equal to n/2.

b) If a number $k > n/2$ divides n, then the other factor would be < 2, which is impossible.

c) If n is even & $n \neq 2$, it's not prime

d) So after checking 2, check only odd numbers.

2. What are the limitations of using int data type for factorial calculation, and how can overflow be handled in C?

A. a) int has a limited range

b) Factorial grows very fast, so values overflow

c) Overflow gives incorrect results without any warning.

d) Use larger data type like long long.

e) Check for overflow before multiplication.

3. How does the time complexity of prime factorization change with increasing input size?

A. a) Using the basic method, prime factorization takes $O(n)$ time if we try all numbers upto n

b) If we check divisors only upto $\sqrt{n}$, the time complexity improves to $O(\sqrt{n})$

(c) As the input size increases, the number of operations grows, rapidly, making factorization slower for large numbers.

4 Why is the math.h library required for square root calculations, and what happens if it is not included?

A. a) The square root function sqrt() is declared in math.h library
b) Including math.h tells the compiler the function defination & return type.
c) The compiler may show a warning or error
d) The program may compile but give incorrect results.

5) How ~~you~~ would the program behaviour change if a negative number is entered for factorial or square root operations?

A. (a) Factorial: factorial is not defined for negative numbers.
(b) Square root: The square root of a negative number is not a real number.

6) Explain the difference between checking a number for primality & generating prime factors of the same num.?

A. a) Checking for primality: Determines whether a number is prime or not. Output is Yes/No.
b) Generating prime factors: Finds all prime numbers that divides the given number.
Output is a list of prime factors.

Date / /

7. How can recursion be used to calculate factorial instead of loops, and what are its advantages & disadvantages?

A. a) Base case: $0! = 1$

b) Recursive case: $n! = n \times (n-1)!$

Advantages: 1) Code is short & easy to understand.

2) Matches the mathematical definition of factorial.

Disadvantages: 1) Uses more memory due to function calls

2) Slower than loops for large values.